EDUDOC AI

Education Document Question Answering System

MINI LLM CAPSTONE PROJECT

Project Domain: Education

Developed Using: Python, LangChain, Ollama, Llama 3.2, ChromaDB and Streamlit

---

ABSTRACT

EduDoc AI is an intelligent document question-answering application developed using Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG).

The application allows users to upload educational PDF documents and ask questions based on their contents. It extracts text, divides it into smaller chunks, generates embeddings and stores them in a ChromaDB vector database.

When a user asks a question, the system retrieves relevant information using semantic search and generates a response using the Llama 3.2 language model through Ollama.

The application provides an interactive interface using Streamlit and supports chat history and CSV downloads.

---

1. INTRODUCTION

Artificial Intelligence is transforming education by making information retrieval easier and faster.

Searching through lengthy PDF documents manually can be time-consuming. EduDoc AI solves this problem by allowing users to ask questions directly about the contents of educational documents.

The application combines document processing, semantic search and Large Language Models to provide relevant answers.

2. OBJECTIVES

- Develop an interactive LLM-based application.
- Extract text from educational PDF documents.
- Generate text embeddings.
- Store embeddings using ChromaDB.
- Implement semantic document search.
- Generate context-based answers using an LLM.
- Develop a user-friendly interface using Streamlit.

- Maintain chat history and provide CSV downloads.

3. TECHNOLOGIES USED

Technology	Purpose
Python 3.12	Programming language
LangChain	LLM application framework
Ollama	Local LLM runtime
Llama 3.2	Language model
ChromaDB	Vector database
Sentence Transformers	Text embeddings
Streamlit	User interface
PyPDF	PDF text extraction
Visual Studio Code	Development environment

4. SYSTEM REQUIREMENTS

Hardware Requirements

- Laptop or desktop computer
- Minimum 8 GB RAM recommended
- Sufficient storage space
- Internet connection for initial downloads

Software Requirements

- Windows Operating System
- Python 3.12
- Visual Studio Code
- Ollama
- Required Python libraries

5. SYSTEM ARCHITECTURE

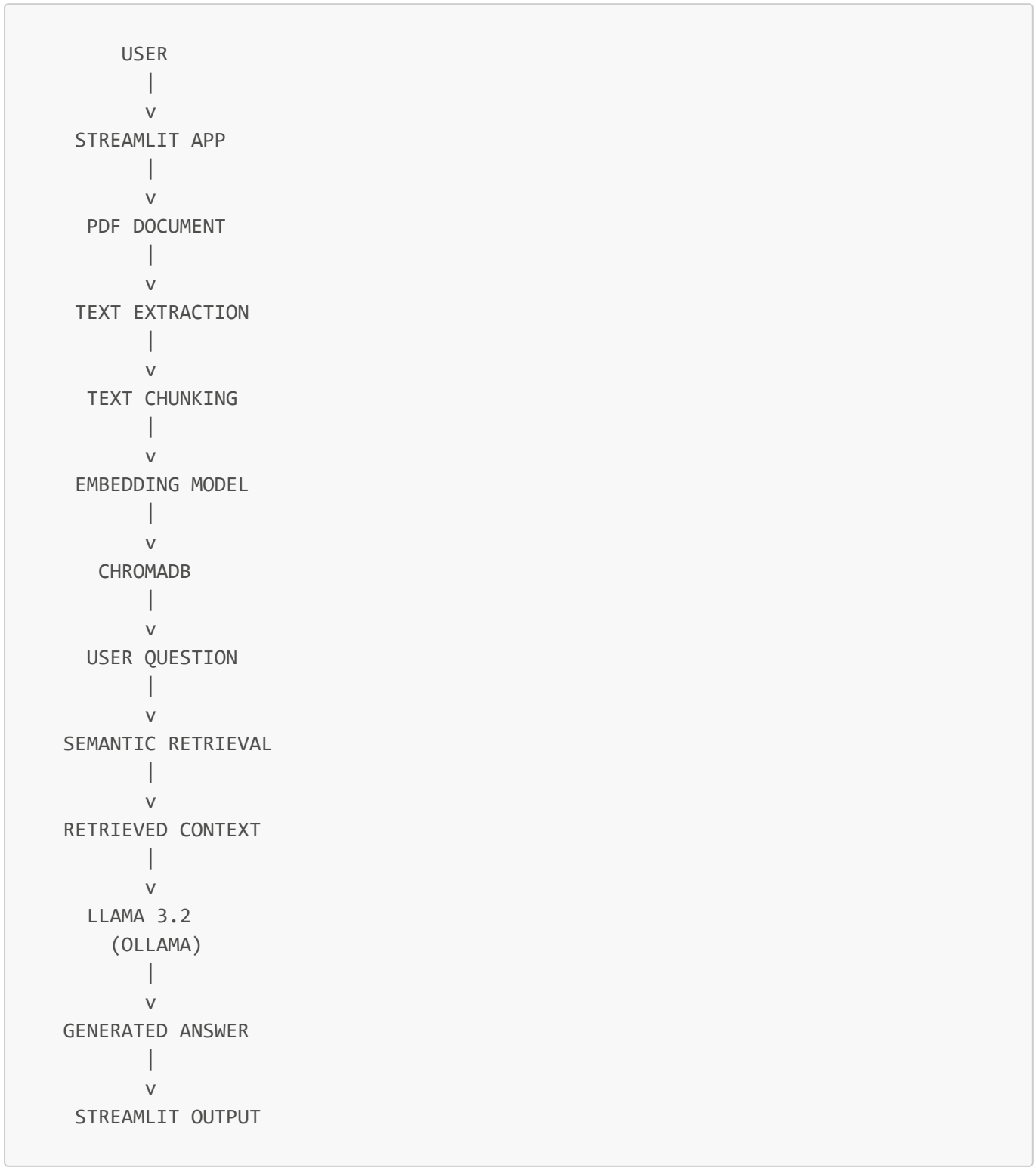
EduDoc AI follows a Retrieval-Augmented Generation architecture.

Workflow

1. User uploads an educational PDF.
2. The system extracts text from the PDF.

3. Extracted text is divided into smaller chunks.
4. Sentence Transformers generates embeddings.
5. Embeddings are stored in ChromaDB.
6. User enters a question through Streamlit.
7. Semantic search retrieves relevant document chunks.
8. Retrieved context is passed to the Llama 3.2 model.
9. The model generates a context-based answer.
10. The answer is displayed in the application.

Architecture Diagram



6. MODULE DESCRIPTION

6.1 PDF Preprocessing Module

The preprocessing module extracts text from PDF documents using PyPDF. It cleans the extracted text and divides it into smaller chunks for further processing.

6.2 Embedding Module

The embedding module uses the Sentence Transformers model `all-MiniLM-L6-v2` to convert text chunks into numerical vector representations.

6.3 Vector Database Module

ChromaDB stores the generated embeddings and document information. It supports similarity-based retrieval.

6.4 RAG Pipeline Module

The RAG pipeline retrieves relevant document chunks based on the user's question and prepares the context for answer generation.

6.5 Chatbot Module

The chatbot module uses the Llama 3.2 model through Ollama to generate answers using the retrieved document context.

6.6 Streamlit Application Module

The Streamlit application provides the user interface. It supports PDF uploads, document indexing, question answering, source display and CSV downloads.

7. PROJECT FILE STRUCTURE

```
Mini_LLM_Project/
|
|-- app.py
|-- chatbot.py
|-- embedding.py
|-- preprocessing.py
|-- rag_pipeline.py
|-- requirements.txt
|-- README.md
|-- EduDoc_AI_Project_Report.md
|
|-- assets/
|   |-- screenshots/
|
|-- chroma_db/
```

```
|
|-- dataset/
|   |-- documents/
|   |   |-- sample.pdf
|   |
|   |-- uploaded_files/
|   |-- answers.csv
|   |-- chat_history.csv
|
|-- notebook/
```

8. IMPLEMENTATION

The project was implemented using Python and LangChain.

First, the educational PDF document was processed to extract its text. The extracted content was divided into smaller chunks.

The Sentence Transformers model generated embeddings for the chunks, which were stored in ChromaDB.

When a user enters a question, the application retrieves relevant information using semantic search. The retrieved context is passed to the Llama 3.2 model through Ollama.

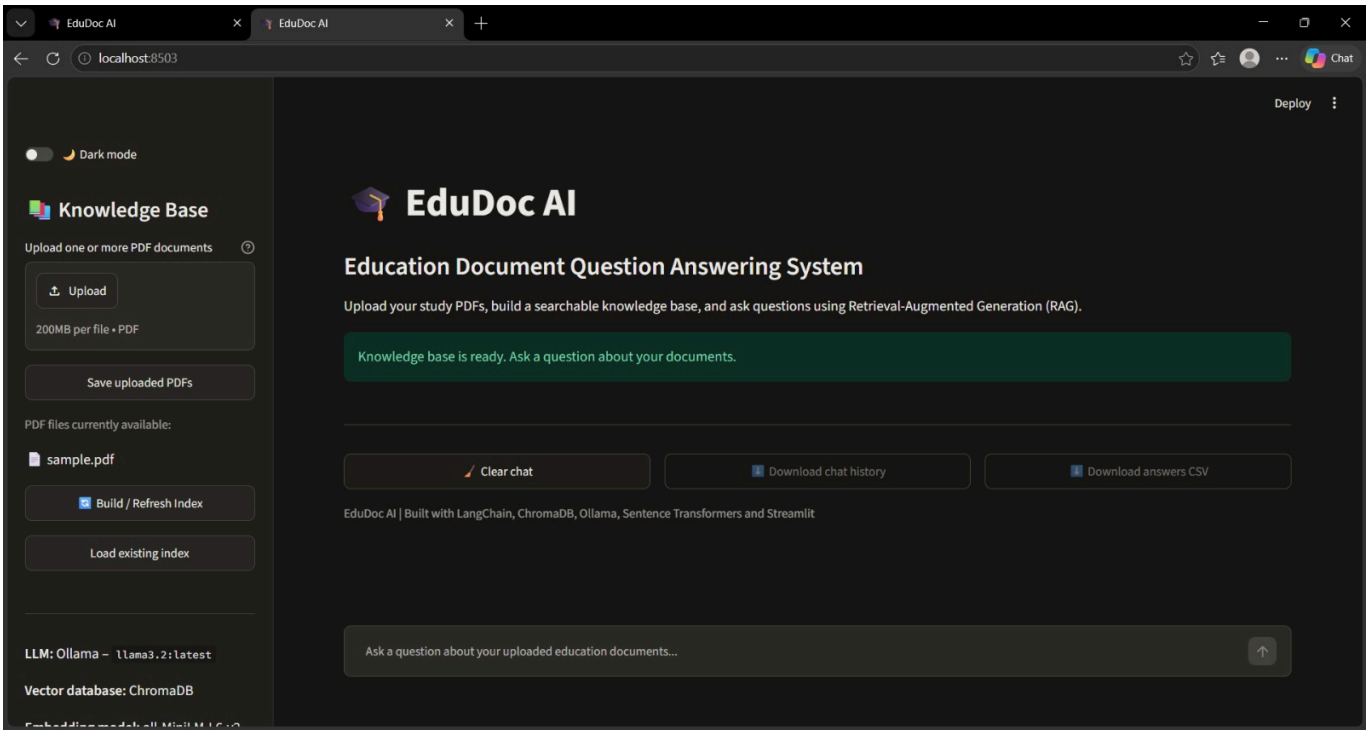
The generated answer is displayed through the Streamlit interface. The application also stores chat history and answers in CSV format.

9. RESULTS AND SCREENSHOTS

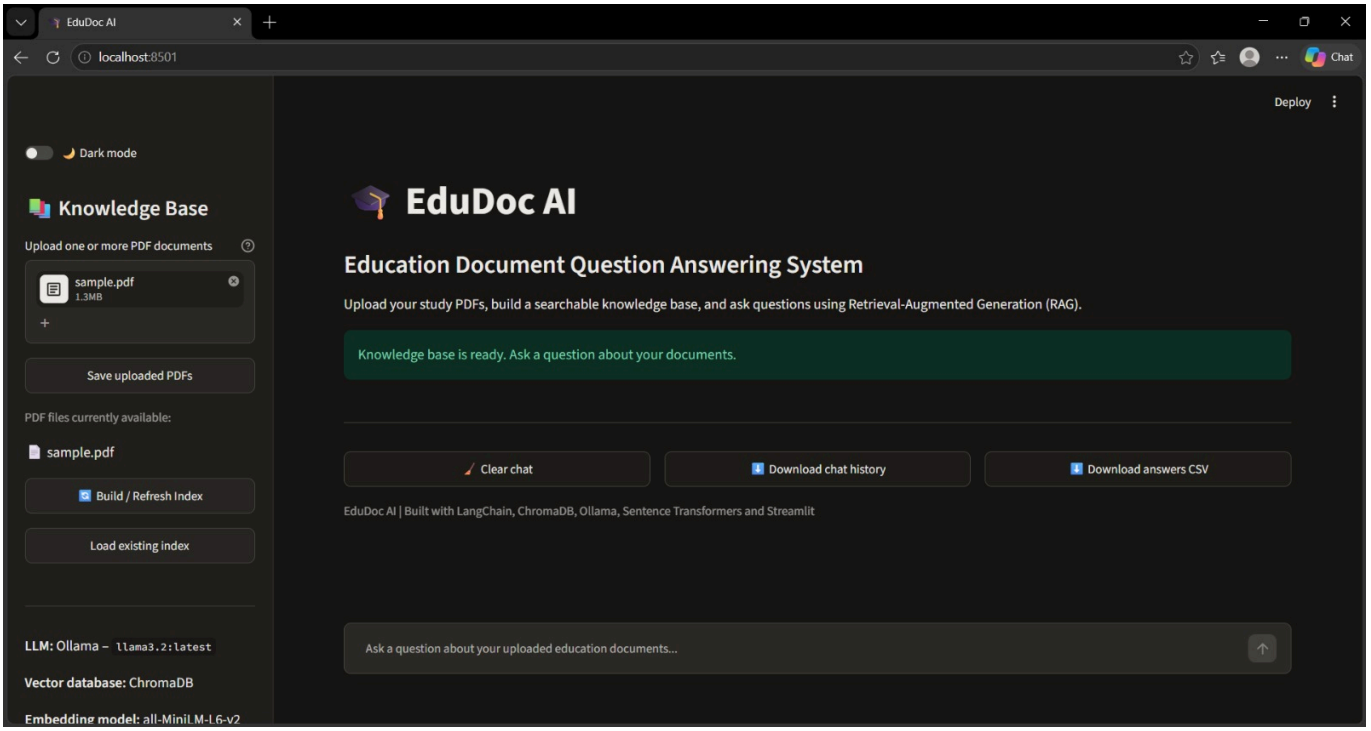
The application was successfully developed and tested.

The following screenshots demonstrate the application features.

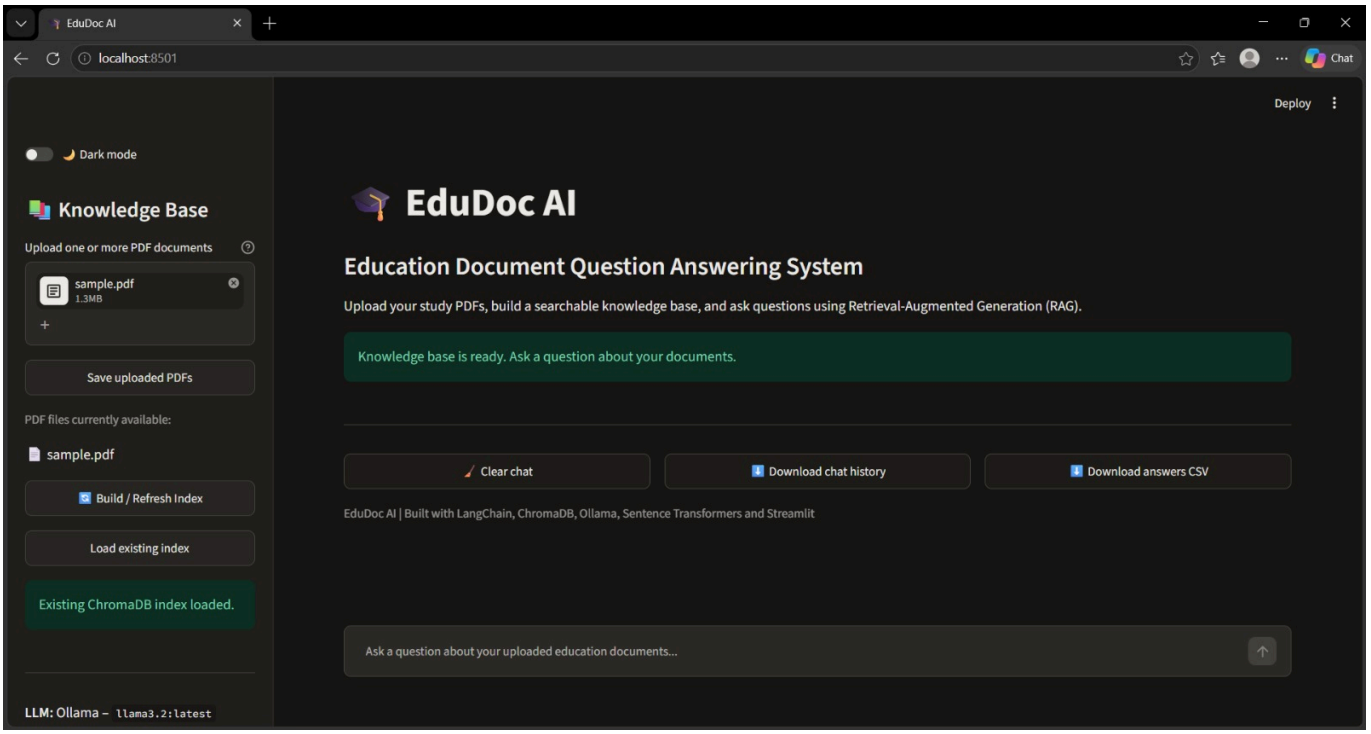
9.1 Application Homepage



9.2 PDF Document



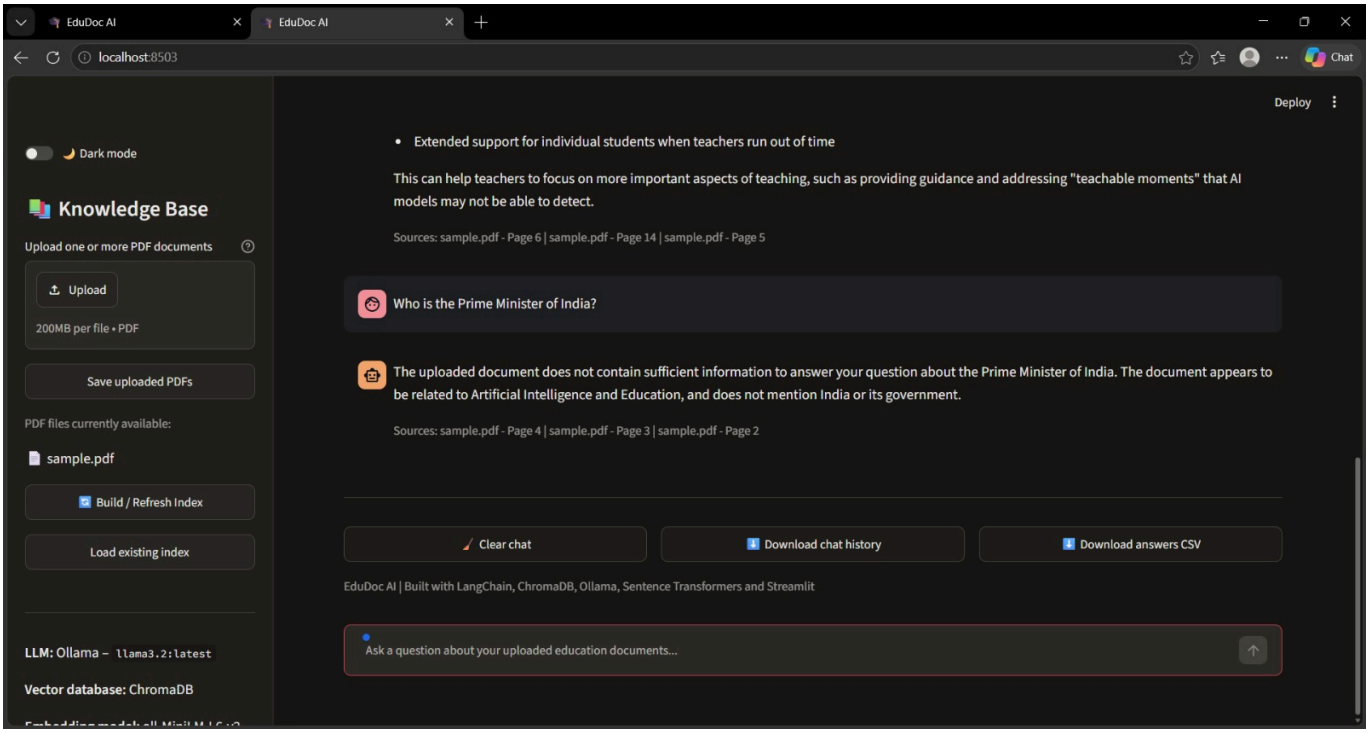
9.3 Document Index



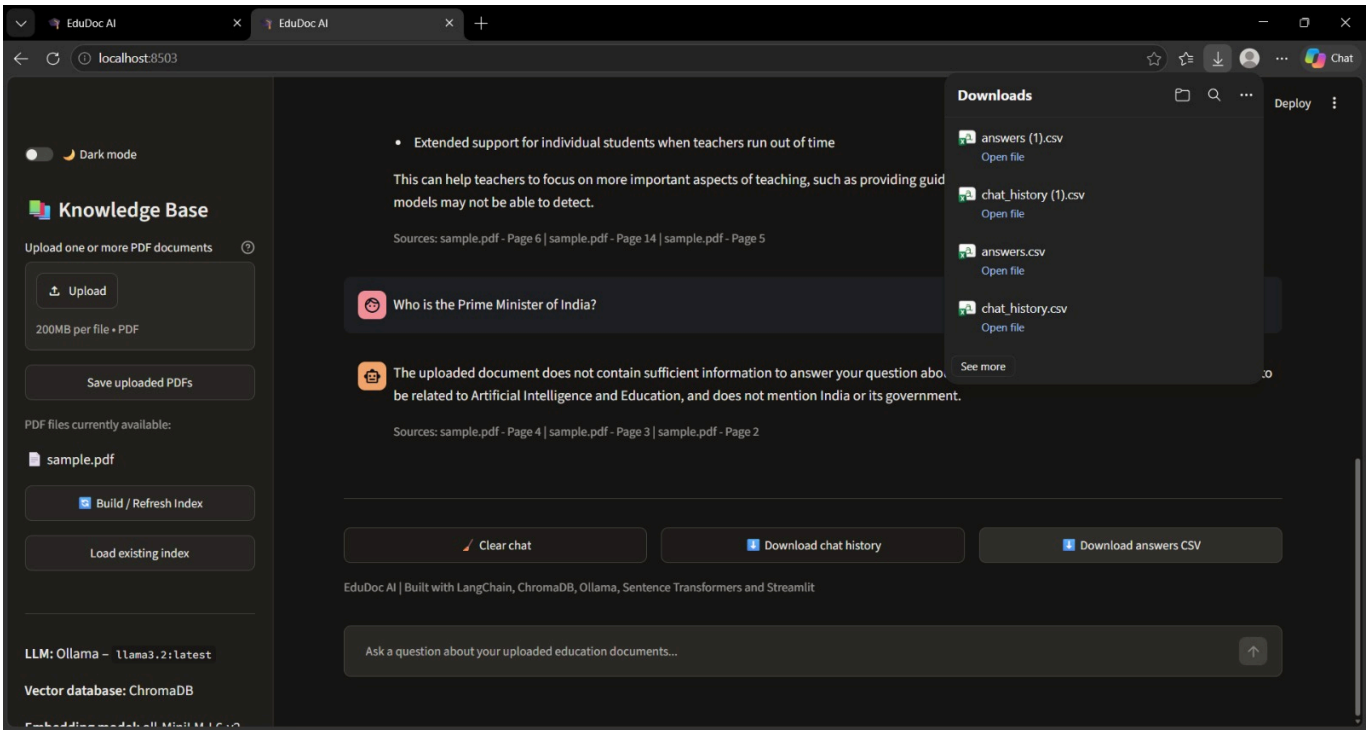
9.4 Chatbot Response – Question 1

Chatbot Response 1

9.5 Chatbot Response – Question 2



9.6 CSV Downloads



10. ADVANTAGES

- Reduces manual searching through lengthy documents.
- Provides quick access to educational information.
- Uses semantic search for relevant information retrieval.
- Runs the language model locally using Ollama.
- Provides an interactive user interface.
- Supports downloadable chat records.

11. FUTURE ENHANCEMENTS

- Support for additional document formats.
- Multilingual question answering.
- Improved document management.
- Enhanced conversational memory.
- Improved source citations.
- Support for multiple educational subjects.

12. CONCLUSION

EduDoc AI demonstrates the practical implementation of Large Language Models and Retrieval-Augmented Generation in the education domain.

By combining LangChain, ChromaDB, Sentence Transformers and Ollama, the application provides an efficient way to retrieve information from educational PDF documents.

The project successfully integrates document preprocessing, semantic search, AI-based answer generation and an interactive Streamlit interface into a single application.

END OF REPORT